

Define Prompt Engineering

Think of LLMs as skilled chefs capable of making amazing dishes with many ingredients. However, just like chefs need a clear recipe to make a dish, language models need well-crafted prompts to produce the desired output.

Prompt engineering is the art of writing a prompt, choosing the right words to guide the model's response, and adjusting them until you get exactly what you want.

A prompt is the instructions given to an LLM to harness its full potential and generate a response. Prompt engineering takes it a step further by carefully crafting and refining these prompts to guide the LLM towards producing the exact output we need.

Writing prompts with clear objectives, goals, and requirements helps the model understand what users want, leading to more relevant and accurate responses. We can transform a simple query into a well-crafted prompt that guides the AI model to understand and generate better responses using various elements of prompts. Think of elements as building blocks that are combined to form an effective prompt.

Elements of prompts

The fundamental elements of prompts are listed below:

Instruction

Context

Constraints

Format

Variables

Instruction

The instruction is the most basic element of a prompt, providing clear and concise guidelines for the model to follow. It directs the model's action, ensuring it performs the desired task with precision and accuracy. The instruction should be straightforward, unambiguous, concise, and focused, allowing the model to generate an accurate response.

An instruction can be of various forms, such as:

Question: "How many months are there in a year?"

Task-oriented statement: "Summarize the following paragraph from Wikipedia..."

A directive to generate content: "Write a poem about a boy who..."

Consider a scenario where an employee, John, arrives at the office on a Monday morning, and his manager informs him that he needs to travel to Dubai for an urgent business meeting with a client. John has never traveled before, so he quickly contacts a travel agency to help him plan the trip. The travel agent, Olivia, decides to use a large language model to create a packing list for the trip. Let's see how the model responds.

Prompt: Create a packing list for a trip

That's perfect. The model successfully created a packing list for John.

Context

Context is any additional information or relevant background related to the query that is necessary for the model to generate an accurate response. Providing context helps the

model respond to queries according to the provided information, guiding it in the right direction to generate a relevant response.

Let's refine the prompt by providing more details about the trip, such as the destination, duration, and type of trip. In doing so, we help LLM generate a more specific packing list that meets John's specific requirements.

Prompt: Create a packing list for a two-day business trip to Dubai.

Now the model will generate a more specific and relevant packing list by providing context, such as the duration, type of trip, and destination.

Constraints

LLMs are powerful machine learning models that can generate unlimited content if they are not properly guided. To overcome this issue, we apply constraints to our prompts.

Constraints are the limitations or restrictions that need to be implemented on the response generated by the model. They specify the scope of the response, such as what to include and what not to include, length, tone, audience, and much more. Constraints are also an important element of the prompt as they allow the model to generate a response within specific boundaries that meet the user's requirements.

Let's try adding some constraints to our prompt, like the audience, weight limit, and the categories of items to pack.

Prompt: Create a packing list for a two-day business trip to Dubai for a 40-year-old man. Consider that the luggage has a strict weight limit of 10kg, and we need to pack professional attire, toiletries, and work items.

Now the model will generate the response within our specified constraints.

Format

There are times when we need a response that is not only accurate but also organized in a specific order or structure. For example, we may need our response in bullets, tables, categories, or any other custom style. This is where the format element comes in.

The format is the structure or layout of our generated response. It is the instruction on organizing, styling, presenting, and meeting any other requirement of a generated response.

The format helps us define the structure of output beforehand in the prompt. Without it, the model will generate random output that needs to be formatted later according to the requirement.

In the business trip scenario, the travel agent got the response from LLM, but this is not enough. She also needs to share the response with John. She can either write an email herself or specify the format of output (email) in her prompt, saving time and effort.

Prompt: Create a packing list for a two-day business trip to Dubai for a 40-year-old man. Consider that the luggage has a strict weight limit of 10 kg, and we need to pack professional attire, toiletries, and work items. Provide the response in the format of an email.

Now the model will generate the response in the form of email with subject, sender name, receiver name etc.

Variables

We can further tweak the prompt to make it more efficient. But before we see how try answering the following questions:

Can the travel agent make LLM write a complete name, removing the minimal effort of writing names?

How can the travel agent efficiently send such emails to multiple customers?

Well, a simple solution that comes to mind is mentioning the names in the prompt and writing a new prompt for each customer according to their trip details, but this would require a lot of effort, right?

How about automating this task? The travel agent can save time and effort by writing a template prompt with variables for trip details. Now, she just needs to change the variable details according to the trip, and the prompt is ready for LLM.

Variables involve using placeholders or flags to customize the prompt. We can specify the variables in our prompt using the `{{variable_name}}`. This is the standard and most widely used format for writing variables, but there are some other formats to provide variables, which include:

Curly braces: `{variable_name}`

Square brackets: `[variable_name]`

Parentheses: `(variable_name)`

Dollar sign: `$variable_name`

Percent signs: `%variable_name%`

Prompt:

`{{Receiver}}` = John

`{{Sender}}` = Olivia

`{{Days}}` = 2 days

`{{TripType}}` = business trip

`{{Destination}}` = Dubai

`{{Weight}}` = 10 kg

Write an email from `{{Sender}}` to `{{Receiver}}` with a packing list for a `{{Days}}` `{{TripType}}` to `{{Destination}}`. Consider that the luggage has a strict weight limit of `{{Weight}}`.

Provide the response in the format of an email.

Now, sending an email is very easy for the travel agent. She will just update the variables with the trip details of the new customer and LLM will generate an email with customized packing essentials for the customer.

Importance of prompt engineering

LLMs have revolutionized the AI landscape with their ability to understand and generate human-like language. The following are the key reasons that highlight the importance of prompt engineering in AI:

Accuracy and relevance: Well-designed prompts optimize interaction with LLMs, leading to more accurate and relevant responses, essential for applications such as text summarization, question-answering, and more.

Performing complex tasks: Prompt engineering enables AI models to perform complex tasks and generate accurate responses, significantly expanding the capabilities of language models.

User experience: Effective prompts enable users to access their required information more quickly and easily, enhancing the overall user experience with LLMs and making them more accessible and beneficial for a diverse range of users.

Ethical considerations: Prompt engineering plays a vital role in ensuring that language models generate responsible and ethical responses that are aligned with human values and carefully avoid biases.

Prompt engineering is a vital component of AI development. As AI continues to transform industries and daily life, mastering prompt engineering becomes a key skill to maximize its potential.

Parameters of a Prompt

Now, we will learn about using parameters in prompts to improve the LLM performance.

Certain parameters can be applied in prompt engineering to help control the model's output. These parameters can improve the model's performance by controlling the output tokens instead of refining the input prompts. The parameters include:

Temperature

Sampling

Repetition penalty

Max tokens

Temperature

The temperature is a key parameter in prompt engineering that controls the randomness and creativity of the generated output. It's a floating point variable ranging from 0 to 2. We can adjust its value to achieve a balance between structured and creative responses.

A temperature value closer to 0 generates a more confident and deterministic response. This setting is useful for scenarios where we need more fact-based and reliable responses, for example, writing a technical blog or performing mathematical calculations.

Moving the temperature value closer to 2 generates more diverse and unpredictable responses. It is best suited for cases where we need creativity, such as writing a story or poem.

Sampling

LLMs work by predicting the probabilities of the next possible tokens and then sampling the highest probability tokens from them to generate the response. There are two primary sampling techniques:

Top-p sampling

Top-k sampling

Top-p sampling

Top-p sampling, also known as nucleus sampling, works by selecting tokens from the cumulative probability distribution until a threshold is met. The value of p sets the threshold for the cumulative probability distribution. The model continues to select the tokens from the probability distribution until the threshold is reached. Then, it randomly samples the tokens from these selected tokens to generate the response.

This parameter controls the diversity and randomness of the output by limiting the token choices for the model. A low top-p value will generate a focused and confident response due to the limited choice of tokens available. A higher top-p value will generate a creative and diverse response because of the wider range of tokens available to the model.

Let's understand this by taking an example of sentence completion and prompt LLM with a lower top-p value of $\text{top_p}=0.2$.

Prompt: Complete the sentence "In the deep, dark forest, the adventurers discovered..."

Response: "...a mysterious, ancient temple hidden behind a waterfall, its entrance guarded by a pair of glowing, ethereal statues with eyes that seemed to pierce through the darkness."

Let's try the same prompt with a higher top-p value of $\text{top_p}=0.8$ and see how the response changes.

Response: "...a clearing surrounded by a ring of mushrooms, their caps aglow with an otherworldly light, and in the center of the clearing stood an enormous, twisted tree, its branches like skeletal fingers reaching towards the moon"

Top-k sampling

Top-k sampling selects the tokens from the top k most likely options in the probability distribution to generate the response. It helps the models avoid including low-probability tokens in the response.

The top-k parameter also controls the diversity and randomness of the output, with a lower value generating a focused response and a higher value generating a creative and diverse output.

Let's understand this by taking an example of sentence completion and prompt LLM with a lower top-k value of top_k=10.

Prompt: Complete the sentence "The treasure chest opened to reveal..."

Response: "...a dazzling trove of glittering jewels, shimmering gold coins, and a map etched on a piece of yellowed parchment, hinting at the location of an even greater treasure hidden deep within the mystical islands of the ancient sea gods."

Let's try the same prompt with a higher top-k value of top_k=60 and see how the response changes.

Response: "...a cascade of sparkling diamonds, a golden amulet adorned with a fiery opal, and a cryptic message scrawled on a piece of velvet, warning of a centuries-old curse that would be unleashed upon anyone who dared to claim the treasure as their own."

Repetition penalty

A repetition penalty parameter controls the probability of generating tokens that have already appeared in the prompt or generated response. The default value of the repetition penalty is 1.0, which means it is disabled. We can set it to a higher value to avoid repeating phrases and words. This parameter is particularly valuable for generating diverse and non-repetitive output.

Let's try prompting the LLM with repetition_penalty=1.1 and see its response.

Prompt: Describe the experience of traveling through a magical portal to another world in one paragraph.

Response: As I stepped through the shimmering portal, the air around me seemed to vibrate with an otherworldly energy, like the hum of a thousand tiny harp strings. The colors of my familiar surroundings began to bleed and blur, replaced by a kaleidoscope of iridescent hues that swirled and danced before my eyes. The sensation was akin to being pulled apart and put back together again, my molecules rearranging themselves to accommodate the fantastical landscape unfolding before me. As I emerged on the other side, I felt the gentle tug of gravity's release, as if I were floating upwards into a realm where the laws of physics were mere suggestions. The air was alive with whispers of ancient magic, and the sky pulsed with a soft, ethereal glow that seemed to emanate from the very fabric of reality itself. Every sense was heightened, every nerve tingling with wonder, as I gazed upon a world that defied explanation, a realm of breathtaking beauty and mystique that beckoned me to explore its secrets.

Now try the same prompt with repetition_penalty=1.6 and see how the response changes.

Response: Stepping through the luminous veil, time warped and space bent, transporting me instantaneously across dimensions; it was both exhilarating terror and thrilling ecstasy all at once! Colors burst forth anew: emerald skies blushed pinkish-golden sunsets; sapphire mountains pierced crystalline clouds while silver rivers flowed uphill towards skies ablaze! Gravity surrendered her hold allowing weightless suspension within swirling vortexes which

dissolved solidity leaving only essence! Whispers echoed within - primeval incantations - harmonizing celestial rhythms resonated within each cell! With senses afire - synesthesia reignited! Sounds became scents becoming tastes! In this ephemeral threshold between worlds past-present-future merged! And when finally deposited onto alien soil – breath caught mid-gasp! For here lay realms beyond mortal comprehension waiting discovery! Ephemera etched memories indelibly! This odyssey birthed anew! Unseen forces awakened! An odalisque unfurled! Unwritten tales awaited! Unbridle curiosity now roamed free! Welcome...to Elsewhere! 🧙🏻‍♂️🌌🌟🌀🥰! (Note : Some emojis are added for extra flair) 😊! 😊! 👍! 🎉! 🥰! (These emojis can be removed according to your preference!) 😊!

Max tokens

The max tokens parameter is the maximum number of tokens a model can generate as an output. The model generates a response within the max tokens limit and automatically truncates the response if it exceeds the limit. This parameter prevents the generation of overly lengthy or irrelevant responses, ensuring efficiency and relevance.

Let's try the same prompt with the max_tokens=100 and see the response.

Prompt: Describe the experience of traveling through a magical portal to another world in one paragraph.

Response: "As I stepped through the shimmering portal, I felt the world around me dissolve into a swirling vortex of color and light. The air seemed to vibrate with an otherworldly energy, and my body felt weightless, as if I were floating on a sea of pure potentiality. The familiar sounds of reality gave way to a symphony of whispers and chimes, echoing off the boundaries of my consciousness. Time itself seemed to bend and warp, stretching out moments into eternity and compressing others"

As we can see, the response is truncated after it reaches the max tokens limit, which is 100.

The longer max tokens value will likely produce a more comprehensive and coherent response. However, this means higher computational and memory usage. On the other hand, opting for a shorter max tokens value may be cost-effective, but it can potentially compromise the reliability and completeness of the generated response.

Prompt Iteration and Evaluation

Sometimes, if we write a complete prompt for LLM using a combination of different prompt engineering techniques with appropriate values of prompt engineering parameters, LLM is still unable to generate the required response. This happens because of differences in the underlying training of the model. Each LLM is trained differently to respond to the query. Sometimes, we get our required response in just one query, and sometimes, we need to prompt the model iteratively to reach the point where we get the desired output.

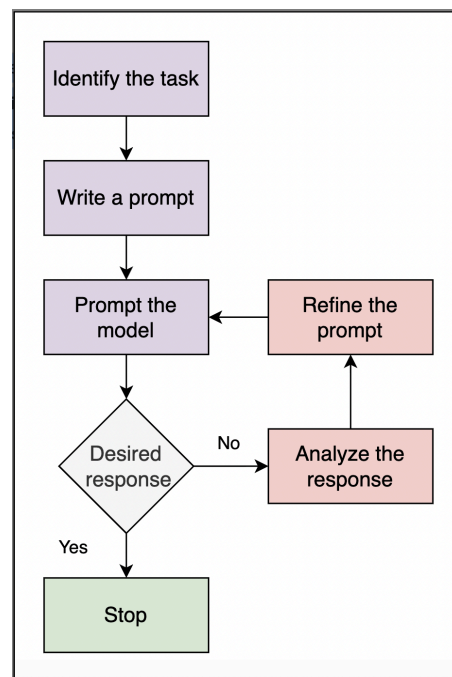
Iterative prompt engineering

Writing a prompt that gives the desired result with accuracy is a challenge in prompt engineering. Iterative prompt engineering is the solution. Iterative prompt engineering is a

process of writing and refining the prompt multiple times until the desired response is generated.

The following steps are involved in iterative prompt engineering:

- **Writing an initial prompt:** The first step is to clearly define the task we want the model to perform. After identifying the task, write an initial prompt with clear and specific instructions using elements of prompts.
- **Prompting the model:** Now, prompt the model with the initial prompt and see how the model responds.
- **Analyzing the response:** Analyze the model response in detail and observe where it lacks our objective. Once we identify the area for improvement, we are ready for refinement.
- **Refining the prompt:** Refine the prompt to address the identified shortcomings and prompt the model. This process is repeated unless we achieve an accurate and relevant response that meets our requirements.



Prompt engineering is an iterative process. The continuous cycle of analyzing the response and refining the prompt is time-consuming, but it significantly improves the model's performance and accuracy.

Evaluating prompt effectiveness

Before prompting the model, we must ensure that we have written a complete and effective prompt. We can evaluate the prompt's effectiveness by evaluating its quality and checking whether it follows all the best practices for prompt writing. We can also evaluate the model response using automated and human evaluation metrics.

Prompt best practices

We can evaluate the prompt's effectiveness by verifying the incorporation of best practices. The following are the best practices that should be part of the prompt.

- A prompt is written with detailed, clear, and specific instructions.
- The necessary context and information are provided to generate the relevant response.
- Specific examples are given about the task.
- The structure, format, and layout of the output are specified in the prompt.
- The complex tasks are broken into subtasks.

Role Prompting

Role prompting is a technique in prompt engineering to control the output generated by the model by assigning a specific role to the model. For example, we can ask the model to act as a poet, a comedian, etc. For effective role prompting, it is essential to give the model some context about how we expect it to respond, given the particular role. The more information and details we give to the model in the prompts, the more chances we get accurate and relevant responses.

Example:

Prompt: You are a fiction writer, create a scene with battle of good and evil.

Prompt:

I want you to act as a motivational speaker. Your goal is to encourage people to take action, overcome obstacles, and achieve personal and professional growth. You should draw from your life experiences, share stories of triumph, resilience, and success to create a connection with your listeners.

Now generate a 200 words speech about never giving up.

Advanced Prompting Techniques

Unlocking the full potential of LLMs requires mastering advanced prompting techniques. These techniques involve writing effective prompts that significantly improve LLM performance. By writing well-structured and well-defined instructions, we can guide the model to generate accurate and relevant responses.

Advanced prompting

The following are some advanced prompting techniques we can use to elevate the model results.

- Zero-shot prompting
- One-shot prompting
- Few-shot prompting
- Chain-of-thought
- Self-consistency
- Role-based prompting

Zero-shot prompting

Zero-shot prompting is the technique of providing simple and direct instructions to the model without any additional context or examples. In this technique, the model is given unseen tasks and uses its training knowledge to generate a response without relying on any previous tasks or examples. In simple words, it is prompting without example, which is why it's also called direct prompting.

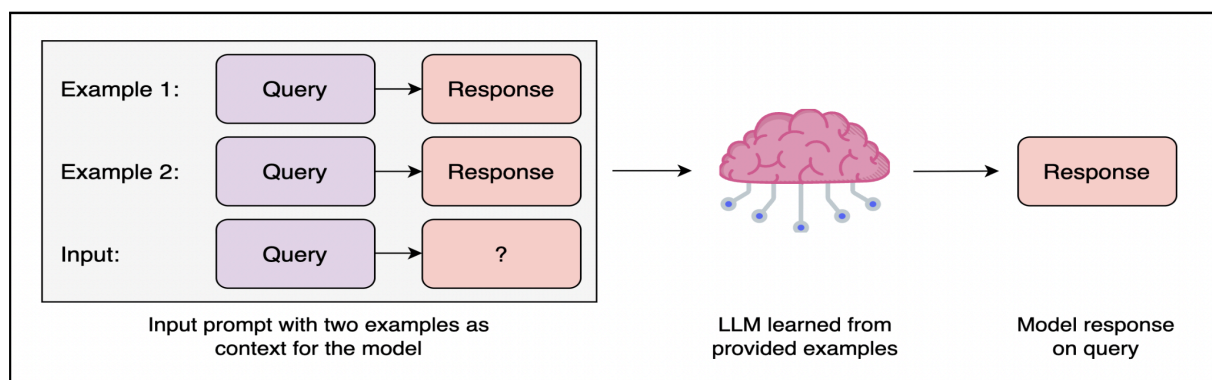
Consider a scenario where an employee, John, arrives at the office on a Monday morning, and his manager informs him that he needs to travel to Dubai for an urgent business meeting with a client. John has never traveled before so he quickly contacts a travel agency to help him plan the trip. The travel agent, Olivia, decides to use the AI model to create a packing list for the trip. Let's see its response with zero-shot prompting.

Prompt: Create a packing list for a two-day business trip to Dubai.

Zero-shot is the easiest and fastest prompting technique, which is more appropriate for simpler tasks. We can use more advanced techniques for complex tasks.

Few-shot prompting

Few-shot prompting is the technique of providing a prompt with a few examples (typically 1–5) as context to the model, guiding it to generate an accurate and desired response. Few-shot prompting is used for relatively complex tasks. The model uses the provided example to understand the pattern and generate a new response by learning from it. This approach is similar to **fine-tuning** the model response but with a much smaller dataset provided as input within the prompt.



Let's apply a **few-shot** prompting by providing an example of a packing list in the form of a table for traveling to San Francisco and see how the model response changes.

Prompt 1: Create a packing list for a two-day conference trip to San Francisco. The trip involves attending formal meetings during the day and casual networking events in the evenings.

Prompt 2: Create a packing list for a two-day business trip to Dubai. The trip involves business meetings during the day and activities like exploring the city and visiting the beach at night.

We can provide one example, called **one-shot prompting**, to n examples in a prompt called **n -shot prompting**. **Few-shot prompting** is not as easy as **zero-shot**, but it makes a huge difference in performance.

Chain-of-thought

Consider solving a complex jigsaw puzzle with thousands of pieces. It seems hard to crack, right? We can solve it by breaking it down into smaller, solvable puzzles. We start by focusing on one section at a time. We find the right pieces and fit them together to solve the smaller puzzle. Once we complete all the small puzzles, we combine them to form a complete picture. This way, we eventually solve the complete jigsaw puzzle. Like solving the jigsaw puzzle, **chain-of-thought (CoT)** prompts tackle complex tasks by dividing them into smaller, logical steps.

Chain-of-thought prompting is a technique of breaking down a task into smaller, sequential reasoning steps. This step-by-step approach enables the model to understand the task more effectively and produce more accurate responses. By providing a series of smaller problems in the prompt, the model can focus on solving one step at a time rather than considering the entire problem at once, leading to more accurate and reasoned responses.

Let's understand this by applying the **chain-of-thought** technique to our problem. We will add a series of steps to our prompt and see how the model responds.

Prompt: John is going on a two-day business trip to Dubai. The trip involves business meetings during the day and activities like exploring the city and visiting the beach at night. Create a packing list for the trip, considering that the luggage has a strict weight limit of 10kg. Follow these steps:

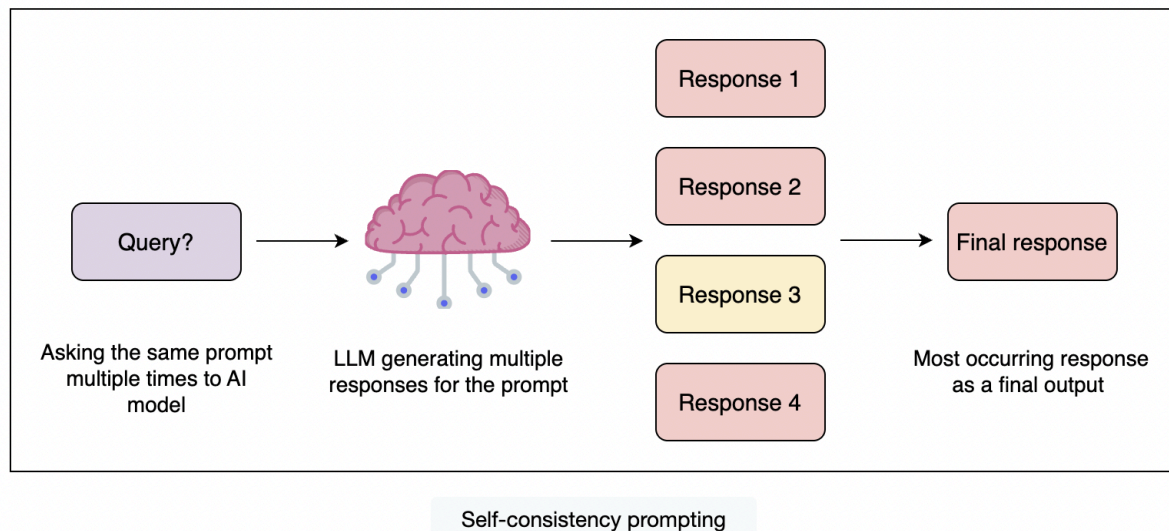
- Consider the weather of Dubai, i.e., hot and identify the categories of items required for the trip.
- List the essential items within each category.
- Provide the quantity and weight of each item.
- Sum the weight of all items at the end of each category.
- Sum the weight of all categories at the end of the list to ensure the total weight does not exceed the 10kg limit.

CoT works well for complex problems such as logical reasoning, arithmetic calculations, and multi-step tasks. Combining it with **zero-shot** and **few-shot** prompting techniques can further enhance the correctness and accuracy of model responses.

Self-consistency

Sometimes, generating a single response isn't enough, and we need multiple thoughts or opinions from different experts to verify its correctness. **Self-consistency** helps us with this.

Self-consistency is an advanced prompting technique that provides the model with the same prompt several times and selects the most occurring response as the final output. This technique gives more reliable and accurate results by generating multiple responses for the same prompt and then choosing the final output through a voting process, where the common response among all the generated responses is selected.



Let's understand this by prompting the LLM with a single prompt multiple times and see its response:

Prompt : John is going on a two-day business trip to Dubai. The trip involves business meetings during the day and exploring the city in the evening. Suggest some city-exploring sites and food points to visit in the evening.

John can select the common suggestions himself or prompt the AI model to extract them from all the responses for city exploration and dining.

Self-consistency is an advanced prompting technique built on the chain of thought. When combined with **CoT**, **self-consistency** significantly improves the accuracy of the model. It's primarily used to enhance model responses for arithmetic calculations, common sense, and logical reasoning tasks.

Role-based prompting

When we want a model to generate a response within a specific domain or character, like a doctor, pharmacist, or professor, role-based prompting comes to the rescue.

Role-based prompting is a technique for controlling the style of generated response by assigning a specific role to the model. For example, we can ask the model to be an author for writing a book or be a poet for writing a poem.

This technique works by guiding the model to respond within the boundaries of the assigned role, resulting in more relevant and context-appropriate responses. Let's understand this by asking the model to act as a travel agent and suggest some beaches or city exploration sites to John for visiting in the evening.

Prompt: Act as a travel agent and suggest some beaches or city exploration sites to visit in the evening on a two-day business trip to Dubai.